

DSST389: Exam I — Reference Sheet

This sheet is attached to your exam. You do not need to memorize any of it.

Setup. Importing `fun`s registers a namespace called `geo` on every Polars DataFrame. Geometries are stored as Well-Known Text (WKT) in a column named `geometry`; the coordinate system is recorded in a column named `_crs`.

Reading and building geometries.

- `fun.read_file(path)` — read a GeoJSON, Shapefile, or GeoPackage into a Polars DataFrame with the geometries converted to WKT.
- `df.geo.points_from_xy(x="lon", y="lat", crs=4326)` — build a point `geometry` column from two coordinate columns and record the CRS.
- `df.geo.to_crs(code)` — reproject the geometries into the given EPSG code.
- `df.geo.explore(column=None, tooltip=["name"])` — interactive map.

Metrics. Some of these are properties and take no parentheses. A property that returns **geometries** overwrites the `geometry` column; one that returns **numbers** appends a new column named after itself.

- `df.geo.centroid` — *property*. Replaces each geometry with its center point.
- `df.geo.area` — *property*. Appends a column `area`, in squared CRS units.
- `df.geo.x` `df.geo.y` — *properties*. Append a column `x` or `y` holding a point's coordinate.
- `df.geo.buffer(distance)` — replaces each geometry with the region within `distance` of it, measured in CRS units. A point becomes a disc.
- `df.geo.count_coordinates()` — appends a column `count_coordinates` giving the number of vertices in each geometry.

Spatial joins. Both tables must be in the same CRS. The output keeps the geometry of the **left** table. Columns whose names appear in both tables get `_left` and `_right` suffixes, and an `index_right` column is added. The join is an **inner** join: left rows with no match are dropped, without an error.

- `df.geo.sjoin(other, predicate="intersects")`
- `df.geo.sjoin_nearest(other, exclusive=False, distance_col=None)` — matches each left row to the closest right row. `exclusive=True` forbids a row from matching itself; `distance_col` names a column to hold the distance, in CRS units.
- Predicates: **intersects** (share any space, including a boundary), **contains** (the first fully encloses the second), **within** (the first is fully enclosed by the second), **touches** (share a boundary but no interior).

Changing the geometries.

- `df.geo.dissolve(by="col", aggfunc={"area": "sum"})` — the spatial counterpart of `group_by` and `agg`. Merges the geometries within each group into one and erases the internal boundaries. The group key comes back as an ordinary column.
- `df.geo.explode()` — one row per part of a multi-part geometry. The other columns are copied onto every part.
- `df.geo.simplify(tolerance)` — drops vertices, keeping the boundary within `tolerance` of its original position, in CRS units. Each geometry is simplified independently, so shared borders can develop gaps: simplify for display, not for analysis.

Plotting. `geom_map` draws the `geometry` column and takes `fill`, `color`, and `alpha` as either fixed values or aesthetics. `coord_fixed` keeps one unit on the x-axis the same length as one unit on the y-axis.

```
(  
  ggplot(state, aes(fill=c.area))  
  .geom_map(color="white")  
  .coord_fixed()  
)
```

Coordinate reference systems. Areas and distances come back in the units of the current CRS. In a meter-based CRS an area is in square meters, and $1 \text{ km}^2 = 1,000,000 \text{ m}^2$.

- **EPSG:4326**, WGS 84 — longitude and latitude, in **degrees**. GPS, raw data, and the default of `points_from_xy`. Never for areas or distances.
- **EPSG:5069**, NAD83 Conus Albers — equal-area, contiguous United States. Units: **meters**.
- **EPSG:3857**, Web Mercator — screen maps and Google Maps. Badly distorts area.
- **EPSG:32618**, UTM Zone 18N — the US East Coast. Units: meters.